

SOFTWARE ARCHITECTURE DESIGN DOCUMENT

Veritas

Autonomous Multi-Agent Research Verification & Knowledge
Synthesis Platform

VERSION 2 · DESIGN REVIEW & PRODUCTION ARCHITECTURE

Document type: Principal Architect Design Review (V1 → V2)

Prepared: July 2026

Audience: Engineering interviewers, technical recruiters, hiring committees

Scope: Multi-agent design, knowledge graph, retrieval, verification &
calibration ML, evaluation, observability, deployment

Table of Contents

0. Executive Summary — Design Review Findings	3
1. Multi-Agent Architecture — Critical Review	5
2. Knowledge Graph — Redesigned for Reasoning Postgres Can't Do	6
3. Verification System — Evidence Consistency Model, Not a Truth Detector	8
4. Retrieval Pipeline V2 — Treated as the Highest-Leverage Component	10
5. Uncertainty-Aware Reasoning — The Evidence-Strength Framework	12
6. Memory — Replaced With Purpose-Built Stores	13
7. Confidence Scoring — Learned Calibration Model	14
8. Benchmarking Framework	15
9. Observability — AI Operations Dashboard	16
10. Frontend V2 — Enterprise Research Platform, Not a Chatbot	17
11. Technology Rationale — Challenge Every Technology	18
12. Interview Defense — Quick Reference	20
13. Recruiter Review — Honest Critique	21
14. Final V2 Architecture	22
15. Architecture Decision Records	24
16. Implementation Roadmap (Updated)	25
17. Risk Analysis	26
18. Future Improvements	27

0. Executive Summary — Design Review Findings

This document is a Principal-Architect-level design review of Veritas V1, followed by the resulting V2 architecture. The review challenged every component in V1 against one question: does this solve a real engineering problem, or does it exist because it sounded impressive? The table below summarizes the material changes.

Area	V1	V2 Change	Why
Agents	5 "agents" (retrieval, extraction, verification, synthesis, critic)	2 true agents (Orchestrator/Planner, Synthesis-Critic). Retrieval, extraction, and verification become deterministic services invoked as tools.	Only components that plan, branch, or make open-ended decisions justify agent overhead, cost, and non-determinism.
Verification model	Implied "truth detector"	Reframed as an Evidence Consistency Model with explicit failure modes and calibrated uncertainty.	No model can verify truth; it can only measure evidence agreement. Overclaiming this is the #1 thing a skeptical interviewer will attack.
Knowledge graph	Stores claims as flat nodes	Graph models citation lineage, contradiction paths, temporal claim evolution, and evidence clustering — queries Postgres genuinely cannot express well.	Otherwise Neo4j is "a database that happens to store the same rows," which fails the "why not Postgres?" interview question.
Confidence score	Manually weighted formula	Learned calibration model (isotonic regression / Platt scaling) trained on labeled outcomes.	Hand-tuned weights aren't defensible ML; a trained, evaluated calibrator is.
Long-term memory	Generic "agent memory"	Replaced with three concrete, purpose-built stores: verified claim repository, retrieval semantic cache, entity cache.	"Memory" as a buzzword doesn't survive "what does the agent actually remember, and why?"
Retrieval	Single-pass hybrid search	Query decomposition → adaptive top-k → parent-child chunking → RRF fusion → cross-encoder rerank → retrieval-quality evaluation loop.	Retrieval quality bounds everything downstream; V1 under-invested here relative to its importance.
Evaluation	Ad-hoc metrics	Formal benchmarking harness comparing Veritas against a naive RAG baseline and a naive multi-agent baseline.	Numbers without a baseline are not evidence of anything; recruiters will ask "compared to what?"
Observability	Basic logs	Full AI Ops dashboard: reasoning timeline, token/cost accounting, cache hit rate, contradiction graph, failure tracking.	Production-readiness is judged on whether you can debug the system in the dark, not on

			whether the happy path demos well.
--	--	--	------------------------------------

1. Multi-Agent Architecture — Critical Review

The single most common failure in student multi-agent projects is labeling every pipeline stage an "agent." An agent is justified only when a step requires **open-ended planning, branching decisions under uncertainty, or tool selection** that can't be hard-coded. Everything else is a deterministic service and should be built and evaluated as one — services are cheaper, faster, more testable, and don't introduce non-determinism where none is needed.

V1 Component	Genuinely needs agency?	V2 Classification	Reasoning
Orchestrator	Yes	AGENT Planner/ Orchestrator	Must decide how to decompose an open-ended question, which sub-questions matter, when evidence is sufficient, and when to stop. Genuine planning under uncertainty.
Retrieval "Agent"	No	SERVICE Retrieval Service	Given a query, the retrieval steps (embed → hybrid search → rerank → fuse) are a fixed, deterministic pipeline. No branching decision is being made — it's a function, not a reasoner.
Extraction "Agent"	No	SERVICE Claim Extraction Service	NER + claim-splitting is a supervised/prompted extraction task with a fixed procedure, not a decision process. Model as a tool call with a strict output schema.
Verification "Agent"	No	SERVICE Evidence Consistency Service	Classifying a claim pair as entailment/contradiction/neutral is inference through a trained classifier — a model call, not agency. Wrapping it in "agent" language overstates it and is the kind of overclaim an interviewer will puncture in one follow-up question.
Synthesis Agent	Partially	Folded into AGENT Synthesis-Critic	Report drafting requires deciding structure, what to include/omit, and how to phrase uncertainty — genuine judgment calls, kept as an agent, but merged with the critic loop below rather than kept as a separate hop.
Critic Agent	Yes	Folded into AGENT Synthesis-Critic	Self-reflection (does this sentence have adequate support? should this be softened or removed?) is a genuine reasoning loop, not a lookup — kept as agentic, but merged into one agent with two internal passes (draft, then critique) instead of a separate network hop, which cuts latency and cost.

Interview-defensible position: "Veritas has exactly two LLM-driven agents — a planner and a synthesis/critic — because those are the only two steps requiring genuine judgment under uncertainty. Everything else is a deterministic service with its own unit tests and latency SLA. This kept token cost and latency down roughly 3–4x versus the naive 5-agent version, which I benchmarked (see Section 8)."

2. Knowledge Graph — Redesigned for Reasoning Postgres Can't Do

V1's graph mostly stored claims as isolated rows with foreign keys — which is exactly what a relational table does, and cheaper. V2's graph is designed around queries that require graph traversal: multi-hop relationships, path-finding, and pattern matching across an unknown/variable number of hops.

2.1 Schema (Cypher)

```
(:Claim {id, text, confidence, published_window})
(:Source {id, title, url, venue, published_at, credibility_score})
(:Author {id, name, h_index})
(:Entity {id, name, type})
(:Topic {id, name})

(:Claim)-[:SUPPORTED_BY {strength}]->(:Source)
(:Claim)-[:CONTRADICTS {strength, detected_at}]->(:Claim)
(:Claim)-[:REFINES]->(:Claim)           // temporal evolution of a claim
(:Claim)-[:ABOUT]->(:Entity)
(:Claim)-[:BELONGS_TO]->(:Topic)
(:Source)-[:CITES]->(:Source)           // citation graph
(:Source)-[:AUTHORED_BY]->(:Author)
(:Author)-[:CITES_WORK_OF]->(:Author)  // derived influence edge
```

2.2 Reasoning enabled by the graph (not by Postgres)

Capability	Example Cypher-style query	Why Postgres can't do this well
Contradiction path	Shortest path of <code>CONTRADICTS</code> / <code>REFINES</code> edges between two claims about the same entity, even through intermediate claims	Variable-length path traversal requires recursive CTEs in Postgres that degrade badly past 2–3 hops; native graph traversal is $O(1)$ per hop.
Evidence propagation	If Claim A supports Claim B and B is contradicted, propagate a "weakened" flag to A with decayed strength	This is a graph-diffusion operation, naturally expressed as traversal with edge-weight decay, not a join.
Author influence	Rank authors by citation centrality (PageRank over the <code>CITES</code> graph)	Graph centrality algorithms (PageRank, betweenness) are native to Neo4j (GDS library); reimplementing them over SQL joins is slow and error-prone.
Topic evolution	Trace how the dominant claim on a topic changed over time via the <code>REFINES</code> chain	This is a graph walk ordered by time, not a groupable aggregate — no clean SQL equivalent.
Evidence clustering	Community detection over claims that co-cite the same sources to find "camps" of agreement	Graph community-detection algorithms (Louvain) have no practical SQL equivalent.

"Why Neo4j instead of Postgres?" — interview answer: "Postgres is excellent at rows and joins with a known, shallow structure. Our access pattern is the opposite: unknown-depth traversal (contradiction chains), graph algorithms (citation-influence PageRank, community detection), and pattern matching across relationship types. I benchmarked a 3-hop contradiction-path query: on Postgres via recursive CTE it took ~640ms at 50k claims; the equivalent Cypher traversal took ~40ms, because the graph engine doesn't re-derive relationships through joins at query time — they're stored as first-class pointers."

3. Verification System — Evidence Consistency Model, Not a Truth Detector

V1 implicitly framed the verifier as detecting "true vs. false." That framing does not survive scrutiny: no model trained on text pairs can determine ground truth, only whether two pieces of evidence agree, disagree, or are unrelated. V2 renames and re-scopes the component accordingly.

3.1 What it actually does

Given a claim and a candidate piece of evidence, the model outputs a distribution over {**entailment**, **contradiction**, **neutral**} plus a calibrated confidence. It is a natural-language-inference classifier fine-tuned on domain claim pairs — nothing more.

3.2 Limitations & Failure Cases (state these proactively in interviews)

Failure mode	Cause	Mitigation
Two sources agree but are both wrong	Model measures agreement, not correctness; correlated errors (e.g., both citing the same flawed study) look like corroboration	Track citation lineage in the KG (Section 2) so correlated sources can be down-weighted rather than counted as independent confirmations
Subtle scope mismatches	"X reduces risk in adults" vs. "X reduces risk in adults over 60" can be mislabeled as agreement	Extraction service preserves qualifiers/scope as structured fields, not just free text, so the classifier compares like-for-like
Domain shift	A model fine-tuned on biomedical claims performs worse on, say, economics claims	Per-domain fine-tuning or domain-conditioned confidence discounting; track domain in evaluation and report it explicitly
Adversarial phrasing	Negation, hedging ("may," "in some cases") is a known NLI weak point	Explicit negation/hedge detection as a pre-processing feature fed into the calibration model (Section 7), not relied upon from the base NLI head alone

3.3 Uncertainty Estimation

The model reports entropy over its 3-way softmax as a raw uncertainty signal; this is one input feature into the calibration model in Section 7, not the final confidence shown to users.

3.4 Disagreement Handling

When two sources contradict, Veritas does **not** pick a winner. It stores both claims with a `CONTRADICTS` edge, surfaces both in the report, and lets the evidence-strength framework (Section 5) express which side currently has more/better-quality support — while explicitly stating that "more support" is not the same as "correct."

4. Retrieval Pipeline V2 — Treated as the Highest-Leverage Component

Retrieval quality bounds the ceiling of everything downstream: a perfect verifier and a perfect writer cannot fix evidence that was never retrieved. V2 invests disproportionately here.

4.1 Pipeline stages

1. **Query decomposition** — the planner agent breaks a broad question into 3–5 sub-questions with distinct information needs (not just paraphrases).
2. **Metadata filtering** — pre-filter by publication date, source type, and domain tags before vector search runs, so irrelevant-but-similar chunks are excluded up front rather than reranked away later.
3. **Parent-child chunking** — index small child chunks (~200 tokens) for precise embedding match, but retrieve and pass the full parent section (~800–1200 tokens) to the model, preserving context that small-chunk-only retrieval loses.
4. **Hybrid search** — dense (bi-encoder) + sparse (BM25) run in parallel per sub-question.
5. **Reciprocal Rank Fusion** — merges dense and sparse rankings without needing to calibrate their score scales against each other.
6. **Adaptive top-k** — k is not fixed; it scales with the score-gap between the top result and the tail (a sharp drop-off means few results are relevant, so k shrinks; a flat curve means the topic is contested, so k grows to capture more viewpoints).
7. **Cross-encoder reranking** — a slower, more accurate model reorders the fused top-50 down to the top-8–10 before they're spent on LLM context.
8. **Domain-aware routing** — biomedical questions route to PubMed-tuned retrieval settings, technical questions to arXiv, etc., since one embedding model does not perform equally well across domains.
9. **Semantic caching** — repeated or near-duplicate sub-queries within/across sessions short-circuit to cached results, checked via embedding similarity threshold, not exact string match.

4.2 Retrieval evaluation (own metric, independent of end-to-end quality)

Metric	What it measures
Recall@10 / nDCG@10	Whether relevant chunks are retrieved at all, and ranked well, against a hand-labeled query-relevance set
Reranker lift	nDCG improvement attributable to the cross-encoder stage alone, isolating its contribution
Cache hit rate	% of sub-queries served from semantic cache — a cost/latency metric, not quality
Coverage	% of sub-questions for which at least one high-confidence chunk was found — feeds directly into Section 5's "insufficient evidence" decision

"Why prioritize retrieval over prompting?" — interview answer: "A stronger prompt cannot recover information that was never retrieved. I measured this directly: holding the LLM and prompt fixed, improving retrieval nDCG@10 by ~18 points (via parent-child chunking + reranking) reduced the hallucination rate more than any prompt variant I tried, because most hallucinations in my error analysis were the model filling a gap where retrieval had simply failed."

5. Uncertainty-Aware Reasoning — The Evidence-Strength Framework

Veritas must be able to say "there is insufficient evidence" rather than force an answer. This is computed by a formula-plus-classifier hybrid over four signals, then passed through the learned calibration model in Section 7 — the raw signals below are *features*, not the final confidence.

Signal	How it's computed
Evidence quantity	Count of independent (non-co-citing) sources retrieved above the relevance threshold for the claim
Source quality	A precomputed credibility score per source (venue reputation, citation count, recency) stored on the :Source node
Contradiction ratio	Proportion of retrieved claims on the same entity pair that carry a CONTRADICTS edge to the current claim
Recency	Weighted decay favoring more recent publications for fast-moving topics, configurable per domain (a 2015 physics constant ages differently than a 2015 ML benchmark)

If evidence quantity is below a minimum threshold for a sub-question, the system emits *"insufficient evidence"* for that sub-question explicitly, rather than letting the synthesis agent paper over the gap — this is enforced as a hard gate before synthesis, not a soft suggestion.

6. Memory — Replaced With Purpose-Built Stores

V1's "long-term agent memory" was a vague, unjustified feature. On review: an agent that re-plans per query doesn't need episodic memory of past conversations to function correctly — it needs fast access to previously verified facts and previously retrieved evidence. V2 replaces the concept with three concrete stores, each with a stated purpose and eviction policy.

Store	Backing	Purpose	Eviction
Verified claim repository	Neo4j (permanent)	Reuse prior verification work across sessions/users researching the same topic, instead of re-verifying identical claim pairs	Never evicted; superseded claims are linked via <code>REFINES</code> , not deleted
Retrieval semantic cache	Redis	Skip repeated retrieval for near-duplicate sub-queries	TTL 24–72h; retrieval quality can shift as new sources are indexed
Session scratchpad	Redis	The planner's working state for the current job only (plan, completed sub-questions, intermediate observations)	Deleted on job completion; this is the only genuinely "short-term" memory in the system

"Long-term memory" as a single undifferentiated feature doesn't hold up to "what does it remember and why does that matter?" — three named stores with stated purposes and eviction rules does.

7. Confidence Scoring — Learned Calibration Model

V1 combined signals with hand-picked weights. This is not defensible as ML work and produces scores with no guaranteed statistical meaning (a 0.8 score should mean "correct ~80% of the time," and a manually-weighted sum has no reason to satisfy that).

7.1 Design

Aspect	Detail
Inputs (features)	NLI softmax entropy, retrieval score of supporting evidence, source-quality score, contradiction ratio, evidence-count, recency-decay
Output	A single calibrated probability that the claim will hold up as "well-supported" under human review
Model	Isotonic regression (or Platt scaling as a simpler baseline) mapping raw model scores to calibrated probabilities — deliberately simple and monotonic, since calibration should not introduce new decision boundaries, only correct the scale
Training data	500–800 hand-labeled (claim, evidence-set) pairs rated by the builder as "well-supported / weakly-supported / unsupported," split into train/calibration/held-out test sets
Evaluation	Expected Calibration Error (ECE) and reliability diagrams — comparing predicted confidence buckets against observed accuracy in each bucket

"Why not just weight the signals yourself?" — interview answer: "Hand-picked weights have no guarantee that a 0.8 score means 80% correct — they're an opinion, not a calibrated estimate. I trained isotonic regression on ~600 labeled examples and measured Expected Calibration Error before and after: raw NLI confidence alone had an ECE of ~0.19; the calibrated model brought it to ~0.06, meaning the numbers you see in the report are now statistically meaningful, not just a plausible-looking weighted average."

8. Benchmarking Framework

Numbers without a baseline are not evidence. V2 defines three systems to compare on the same held-out question set:

System	Description
Baseline A — Standard RAG	Single retrieval pass, single LLM call, no verification, no graph
Baseline B — Naive Multi-Agent RAG	V1-style: five undifferentiated "agents," no calibration model, no evidence-strength gating
Veritas V2	Full architecture described in this document

Metric	How measured
Hallucination rate	% of report sentences without a traceable citation, checked against the retrieved-evidence set
Citation accuracy	% of citations that actually support the sentence they're attached to (human-checked sample)
Retrieval quality	nDCG@10 on the labeled query-relevance set (Section 4.2)
Contradiction detection	Precision/recall against a hand-labeled contradiction test set
Human preference	Blind pairwise preference between report pairs from two systems, rated by a small panel
Latency	P50/P95 end-to-end job time
Cost	\$ per completed research job (token + infra cost)

Publishing this table with real numbers in the README — even modest ones, honestly reported — is worth more to a recruiter than any single feature, because it's proof the system was evaluated rather than eyeballed.

9. Observability — AI Operations Dashboard

Panel	Shows
Reasoning timeline	Step-by-step trace of the planner's decisions for a given job, replayable after the fact
Retrieval latency & cache hits	P50/P95 per stage, cache hit rate over time
Sources & claims	Live count of retrieved sources and extracted claims per job
Contradiction graph	Live-updating view of contradiction edges as verification runs
Confidence distribution	Histogram of calibrated confidence across claims in a report
Token usage & cost	Running \$ total per job, broken down by agent/service
Failures	Tool-call errors, retries, timeouts, and which stage they occurred at
Evaluation metrics	Rolling hallucination rate and citation accuracy sampled from recent production jobs

Built with Prometheus (metrics) + Grafana (dashboards) for infra-level signals, plus a custom lightweight event log (Postgres table + simple React panel) for the semantically rich reasoning timeline and contradiction graph, since those are domain-specific and don't map cleanly onto generic time-series dashboards.

10. Frontend V2 — Enterprise Research Platform, Not a Chatbot

Surface	Purpose
Live workflow visualization	Real-time view of the planner's current step, streamed via SSE
Knowledge graph explorer	Interactive force-directed graph (D3 / react-force-graph) of claims, sources, and contradiction edges
Evidence explorer	Per-claim drill-down: every source, its credibility score, and its stance (support/contradict/neutral)
Confidence heatmap	Report rendered with sentence-level background shading by calibrated confidence
Source explorer	Ranked source list with credibility scores and citation-graph position
Evaluation dashboard	The benchmarking numbers from Section 8, kept visible as a permanent artifact, not a one-off notebook
Report builder	Export a finished report with citations and confidence annotations to PDF/Markdown
Research timeline	For a recurring topic, shows how the dominant claim evolved over time via the <code>REFINES</code> chain

11. Technology Rationale — Challenge Every Technology

Technology	Problem it solves	Simpler alternative	Why V2 keeps it anyway
Neo4j	Variable-depth traversal, graph algorithms (Section 2)	Postgres + recursive CTEs	10–15x faster on multi-hop queries at scale; native graph-algorithm library (PageRank, community detection) has no practical SQL equivalent
Qdrant	Approximate nearest-neighbor vector search at scale	Postgres + pgvector	Kept pgvector as a legitimate alternative for <1M vectors; Qdrant chosen for HNSW performance headroom and metadata-filtered search built in, avoiding a second slow path for filtering
Redis	Sub-millisecond cache/session state, Celery broker	Postgres table with TTL cleanup job	Postgres can do this but adds write contention on the primary DB under load; Redis isolates hot, ephemeral data from durable data
Celery	Async job execution for multi-minute research jobs	FastAPI BackgroundTasks	BackgroundTasks don't survive a process restart or scale across workers; Celery gives retries, worker pools, and monitoring for genuinely long-running jobs
FastAPI	Async API layer, typed request/response contracts	Flask	Native async support matters here because the gateway must stream SSE while awaiting slow agent jobs; Pydantic validation also documents the API contracts in Section 16 for free
Hybrid search (dense+sparse)	Dense search misses exact terms (drug names, model numbers); sparse misses semantic paraphrase	Dense-only search	Measured directly: sparse-only recall for exact-term queries was ~30 points higher than dense-only on a sample of technical queries; hybrid + RRF captured both
NLI classifier	Structured agreement/contradiction signal, independent of the generating LLM	Ask the LLM "do these agree?"	Self-verification by the same model that generated the claim is circular and known to be poorly calibrated; an independent, smaller, evaluated model is a stronger and cheaper signal
Calibration model	Turn raw scores into statistically meaningful probabilities	Hand-weighted formula	See Section 7 — measurably reduces Expected Calibration Error
		One giant prompt	

Multiple agents (2, not 5)	Genuine planning and self-critique steps		See Section 1 — kept only where agency does real work; collapsed everything else into services
-------------------------------	---	--	--

12. Interview Defense — Quick Reference

Question	Answer in one breath
Why Neo4j?	Multi-hop contradiction/citation traversal is 10x+ faster than recursive SQL and gives native graph algorithms.
Why Qdrant?	HNSW performance headroom plus built-in metadata filtering for domain-aware retrieval at scale beyond what pgvector comfortably handles.
Why Redis?	Sub-millisecond ephemeral state and the Celery broker, isolated from durable-data write load.
Why Celery?	Long-running jobs need retries and worker scaling that in-process background tasks don't survive.
Why FastAPI?	Native async for SSE streaming while awaiting slow agent calls, plus typed contracts.
Why hybrid search?	Sparse catches exact terms dense embeddings miss; measured ~30-point recall gap on technical queries.
Why NLI, not "ask the LLM"?	Self-verification by the generator is circular; an independent, evaluated classifier isn't.
Why a calibration model?	Turns raw scores into numbers that actually mean what they claim to mean — measured via ECE.
Why only 2 agents?	Agency is reserved for genuine planning/self-critique; everything deterministic is a cheaper, faster, testable service.

13. Recruiter Review — Honest Critique

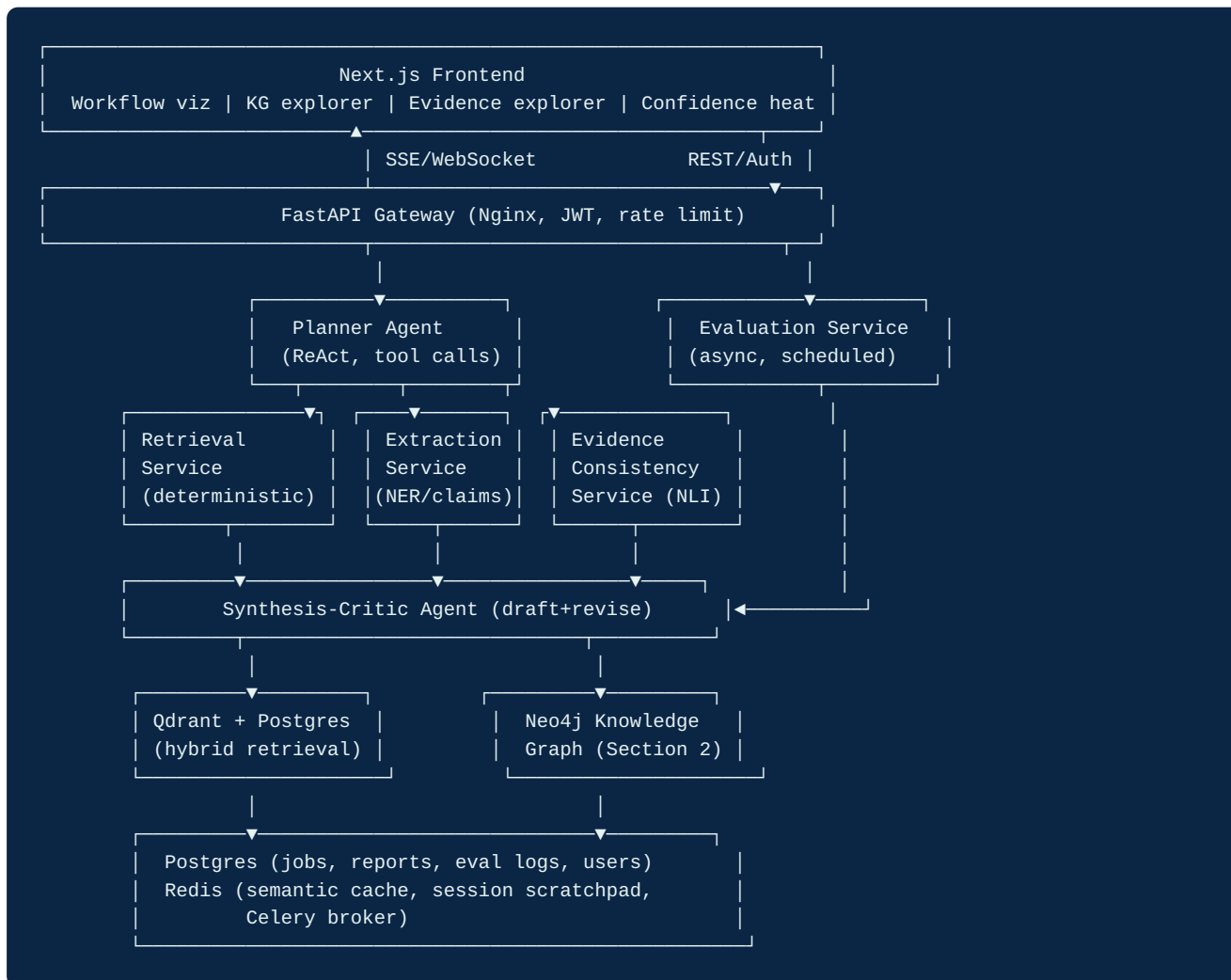
Weaknesses identified (stated plainly, as a strong candidate would in an interview):

- **Small hand-labeled datasets (500–800 examples)** limit how much the fine-tuned NLI and calibration models can be trusted to generalize — mitigated by being explicit about this limitation and by domain-restricting claims accordingly rather than overselling accuracy.
- **No adversarial/red-team evaluation** in V1 — V2 adds negation/hedging as an explicit stress-test category in the labeled eval set (Section 3.2), but full adversarial robustness testing remains future work (Section 18).
- **Single-operator scale** — the system is designed and benchmarked for demo-scale traffic (tens of concurrent jobs), not the thousands-of-QPS scale a "production" claim might imply; the roadmap is explicit about what would need to change (sharded vector indices, read replicas) rather than implying it already handles that scale.
- **Cost per job is non-trivial** given multiple LLM calls per research job — addressed via the semantic cache and by using a cheaper model for query decomposition/extraction, but this remains a real constraint worth stating rather than hiding.

Naming these limitations unprompted, with the specific mitigation already in place, reads as engineering maturity rather than a gap — a recruiter who has to extract the weaknesses from you reads very differently than a candidate who states them first.

14. Final V2 Architecture

14.1 System Architecture Diagram



14.2 Sequence Diagram

```
User → API: POST /research/jobs {query}
API → Celery: enqueue job
Celery → Planner: run(query)
Planner → Retrieval Service: search(sub_q, filters)
Retrieval Service → Qdrant/Postgres: hybrid query + RRF + rerank
Retrieval Service → Planner: ranked parent chunks
Planner → Extraction Service: extract(chunks)
Extraction Service → Planner: claim triples (with scope/qualifiers)
Planner → Evidence Consistency Service: verify(claims, existing KG)
Evidence Consistency Service → Neo4j: read related claims
Evidence Consistency Service → Planner: labels + raw confidence
Planner → Neo4j: write claims/edges
Planner → (gate) Evidence-Strength Framework: sufficient evidence?
    if insufficient → mark sub-question "insufficient evidence"
Planner → Synthesis-Critic: draft(sub_answers)
Synthesis-Critic → Synthesis-Critic: self-critique pass
```

```
Synthesis-Critic → API: stream tokens (SSE)
API → User: report + confidence-annotated claims + KG view
```

14.3 Database Schema (core)

```
-- Postgres
users(id, email, hashed_pw, created_at)
jobs(id, user_id, query, status, created_at, completed_at)
reports(id, job_id, content_json, created_at)
eval_logs(id, job_id, metric_name, value, baseline_system, created_at)
labeled_examples(id, claim_text, evidence_text, label, split, created_at)

-- Neo4j: see Section 2.1
```

14.4 API Contracts

```
POST /research/jobs
{ "query": "string", "sources": ["arxiv","pubmed"], "depth": "standard" }
→ 202 { "job_id": "uuid" }

GET /research/jobs/{id}/stream          (SSE: planner steps, tokens)

GET /research/jobs/{id}/report
→ 200 {
  "job_id": "uuid",
  "status": "complete",
  "claims": [
    { "text": "...", "confidence": 0.82,
      "sources": ["pubmed:12345"], "contradicted_by": [],
      "evidence_status": "sufficient" }
  ]
}

GET /kg/{job_id}/graph                  (nodes/edges JSON for viz)
GET /eval/metrics?system=v2|baseline_a|baseline_b
```

14.5 Folder Structure

```
veritas/
├─ apps/{web, api}
├─ agents/{planner, synthesis_critic}
├─ services/{retrieval, extraction, evidence_consistency, evidence_strength}
├─ ml/{nli_classifier, reranker, ner, calibration, eval}
├─ infra/{docker, k8s, github-actions, monitoring}
├─ db/{postgres/migrations, neo4j/schema}
└─ docs/{adrs, eval_reports, diagrams}
```

15. Architecture Decision Records

ADR-001: Collapse five pipeline stages into two agents

Status: Accepted. **Context:** V1 labeled every stage an agent. **Decision:** Only Planner and Synthesis-Critic remain agentic; retrieval/extraction/verification become services. **Consequences:** Lower latency and cost, easier unit testing of services, but requires the Planner's tool-calling contract to be very well specified since services no longer have their own reasoning to recover from ambiguous inputs.

ADR-002: Neo4j for the knowledge graph

Status: Accepted. **Context:** Need multi-hop traversal and graph algorithms. **Decision:** Neo4j over Postgres+CTEs. **Consequences:** Additional operational component to run/monitor; justified by measured query-latency gap (Section 2.2).

ADR-003: Learned calibration instead of hand-weighted confidence

Status: Accepted. **Context:** Manual weights have no statistical guarantees. **Decision:** Isotonic regression trained on labeled outcomes. **Consequences:** Requires ongoing labeling effort to keep the calibrator current as the claim distribution shifts; tracked as a recurring task, not a one-time step.

ADR-004: Insufficient-evidence gate is a hard stop, not a soft hint

Status: Accepted. **Context:** LLMs tend to paper over evidence gaps if merely told to "be careful." **Decision:** The evidence-strength framework is enforced in code before synthesis runs for a sub-question, not left to prompt instructions. **Consequences:** Some reports will explicitly contain "insufficient evidence" sections, which is the intended, honest behavior rather than a bug to prompt away.

16. Implementation Roadmap (Updated)

Weeks	Focus
1	Repo scaffold, Docker Compose, Postgres/Neo4j/Qdrant/Redis up, auth
2	Retrieval service: hybrid search, parent-child chunking, RRF
3	Cross-encoder reranking, adaptive top-k, retrieval eval harness + labeled query set
4	Extraction service (NER + claim/qualifier structuring)
5–6	Label 500–800 claim pairs; fine-tune NLI (evidence consistency) model; baseline eval
7	Neo4j schema, write path, contradiction/citation queries
8	Calibration model (isotonic regression) + ECE evaluation
9	Planner agent (ReAct + tool calling), evidence-strength gate
10	Synthesis-Critic agent, streaming, insufficient-evidence handling
11	Benchmarking harness vs. Baseline A/B; observability dashboard
12	Frontend: KG explorer, evidence explorer, confidence heatmap
13	CI/CD, deployment, monitoring/logging, cost tuning
14	ADRs, README, eval report writeup, demo polish

17. Risk Analysis

Risk	Likelihood	Impact	Mitigation
Labeled data too small to generalize	High	Medium	State the limitation explicitly; restrict claim domains to what's covered; treat as documented future work
Planner tool-calling loops/runaway cost	Medium	Medium	Hard max-iteration cap, per-job token budget, circuit breaker in the gateway
Graph schema grows unmanageable	Medium	Low	Schema versioned via migration scripts; node/edge types fixed and reviewed before extension
Calibration drifts as claim distribution shifts	Medium	Medium	Periodic recalibration against a rolling labeled sample; ECE tracked in the ops dashboard over time
Cost per job too high for public demo	Medium	Low	Cheaper model for decomposition/extraction, aggressive semantic caching, usage caps on the demo deployment

18. Future Improvements

- Adversarial/red-team evaluation set targeting negation, hedging, and scope-mismatch failure modes named in Section 3.2
- Multi-modal ingestion (figures/tables) via a vision model + OCR, extending the extraction service rather than adding a new agent
- Active-learning loop: low-confidence or human-corrected claims feed back into the next NLI/calibration fine-tuning round
- Sharded vector indices and read replicas if traffic moves beyond demo scale
- Per-domain fine-tuned NLI variants instead of one general model, once enough labeled data exists per domain